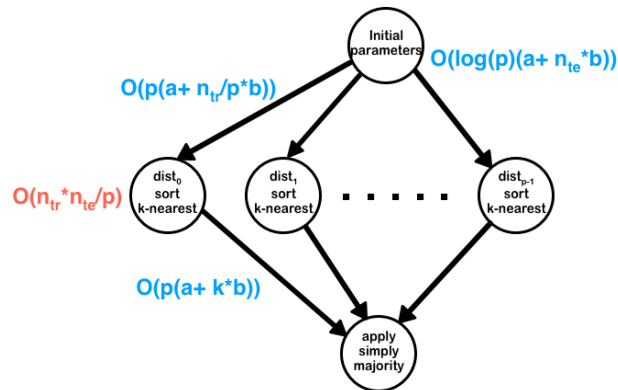


Indicaciones generales:

- se presenta un conjunto de proyectos para que el estudiante escoja aquel que va a desarrollar
 - El proyecto se desarrollará en grupos de 2 o 3 estudiantes. Los **nombres** de los integrantes deberán aparecer al inicio del **informe de proyecto**
 - Los archivos de proyecto se subirán directamente a Canvas (**Proyecto HPC**)
 - La entrega parcial del proyecto será hasta el **domingo 14 de junio**. La entrega final hasta el **viernes 17 de julio**
 - La presentación final y discusión sobre el proyecto será el **sábado 18 de julio** en clase
 - La sección 8 describe **Indicaciones importantes para el proyecto**, y la sección 9, la **Rúbrica**
-

1 KNN

KNN es un algoritmo de clasificación supervisada (aunque también puede aplicarse a regresión) que clasifica un punto nuevo según las etiquetas de sus k vecinos más cercanos en el espacio de características. En paralelo, cada elemento de testeo puede ser calculado independientemente del resto con la fracción de la data de prueba que tiene cada proceso.



Paralelice el código adjunto *knn_digits_sec.py* ^[1], que utiliza la conocida librería *load_digits* para clasificar dígitos.

El desarrollo y **análisis de resultados** debe contener lo siguiente:

- Elabore el código en Python incluyendo las directivas de comunicación (`comm.bcast`, `comm.scatter`, `comm.gather`) ^[2] de acuerdo al criterio mostrado en el DAG
- Registre el desarrollo del código en por lo menos 3 pasos (versiones beta). En caso utilice una fuente externa, mostrar la validación del código
- Mida los tiempos de **ejecución**, **cómputo** y **comunicación** (representélas gráficamente), así como la **precisión** del modelo (accuracy). Incremente tanto procesos (p) como datos (n). Utilice la complejidad teórica para **validar** el resultado obtenido. Describa como normaliza la expresión teórica para que pueda aproximar los experimentos. ¿Cuál estima, sería la cantidad **óptima de procesos**?
- Derive la gráfica de **speedup** de los datos anteriores y genere una gráfica de **FLOPs vs. p** (FLOP por segundo). Obtenga los FLOP de la región paralelizable (entrenamiento), basada en la fórmula de la **distancia euclidiana entre dos puntos**.
- Analice la escalabilidad del algoritmo con un tamaño variable del problema

Bibliografía:

[1] <https://drive.google.com/file/d/1wTBSH7bwT1QVvce0pFMv5tcCWsg4-HeK/view?usp=sharing>

[2] <https://mpi4py.readthedocs.io/en/stable/>

2 TSP (Travelling Salesman Problem)

El Problema del Viajante (TSP) plantea lo siguiente: ¿Cuál es la ruta más corta posible que permite a un viajante visitar un conjunto de ciudades una sola vez y regresar a la ciudad de origen?.

Es un problema clásico de optimización NP-difícil, lo que significa que no hay una solución eficiente conocida para todos los casos. Muchos problemas en ciencia de datos requieren encontrar la mejor solución entre muchas posibles (por ejemplo: rutas, horarios, combinaciones). Ello deriva en aplicaciones directas como:

- Logística y entrega de productos (Amazon, Rappi, FedEx)
- Rutas de drones y robots autónomos
- Secuenciación de genomas (ordenar fragmentos de ADN)
- Diseño de circuitos electrónicos, conexión satelital, en fibra óptica

En grandes volúmenes de datos, el orden en que se procesan tareas puede modelarse como un problema similar al TSP. Esto mejora la eficiencia de: Pipelines de datos, Procesamiento distribuido

El código adjunto⁽¹⁾ resuelve el problema del agente viajero en paralelo utilizando el método *branch and bound* discutido en clase

El código aplica el algoritmo a matrices de distancia entre ciudades (ver data adjunta)

El proyecto debe contener lo siguiente:

- a) Elabore un PRAM del algoritmo y determine la complejidad teórica del mismo
- b) Ejecute mediciones de tiempo con el código y compárelas con la teórica, para distinto número de procesos y tamaño del problema
- c) Desarrolle un software de análisis de performance que permita obtener métricas de performance. El código debe generar la representación gráfica de las métricas y llegar a conclusiones sobre la escalabilidad del mismo.

Bibliografía

- [1] TSP code

3 Expresiones regulares

Expresiones regulares (regex) son una forma de representar un conjunto de cadenas de caracteres que satisfacen ciertas reglas de construcción (gramáticas). E.g. dado el alfabeto $\Sigma = \{0, 1\}$, la expresión regular $01(01)^*$ acepta cadenas como $\{01, 0101, 010101, \dots\}$. Expresiones regulares se usan frecuentemente en algoritmos de manipulación de strings, de análisis lexicográfico y sintáctico, entre otros. Un ejemplo de ello es NetKat, un lenguaje de programación de redes [1]

En Ciencia de Datos, regex es un poderoso utilitario que sirve para manipular patrones en texto, en operaciones de búsqueda, matcheo, reemplazo, splitting, extracción de datos, entre otros.

Un software de reconocimiento de expresiones regulares es de mucha utilidad, especialmente si es escalable. PAREM [2] es un ejemplo de un software de expresiones regulares rápido y escalable, con un speedup lineal con respecto al número de procesos. Este algoritmo particiona la cadena (input), para luego combinar los resultados parciales obtenidos (ver paper).

Se adjunta en el proyecto, una versión de PAREM ⁽¹⁾, desarrollada con OpenMP (visto en clase)

El proyecto debe contener lo siguiente:

- a) Elabore un PRAM del algoritmo y determine la complejidad teórica del mismo
- b) Ejecute mediciones de tiempo con el código y compárelas con la teórica, para distinto número de procesos y tamaño del problema
- c) Desarrolle un software de análisis de performance que permita obtener métricas de performance. El código debe generar la representación gráfica de las métricas y llegar a conclusiones sobre la escalabilidad del mismo.

Bibliografía:

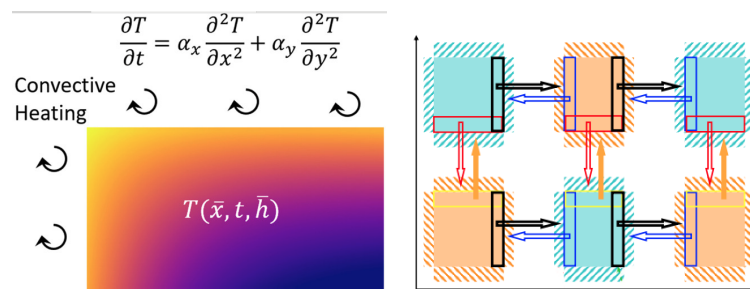
[1] PAREM code

[2] arxiv.org/pdf/1412.1741.pdf

4 Ecuación de calor

Es un caso de solución de un sistema de ecuaciones diferenciales parciales para resolver la ecuación de transferencia de calor. Se utiliza Machine Learning para hacer proyecciones en transferencia de calor, útil para manufactura y aplicaciones en ingeniería.

En este proyecto debe realizar un análisis de performance con el código MPI (adjunto)⁽¹⁾ que resuelve la ecuación de calor (EDP, vista en clase) en paralelo.



El proyecto debe contener lo siguiente:

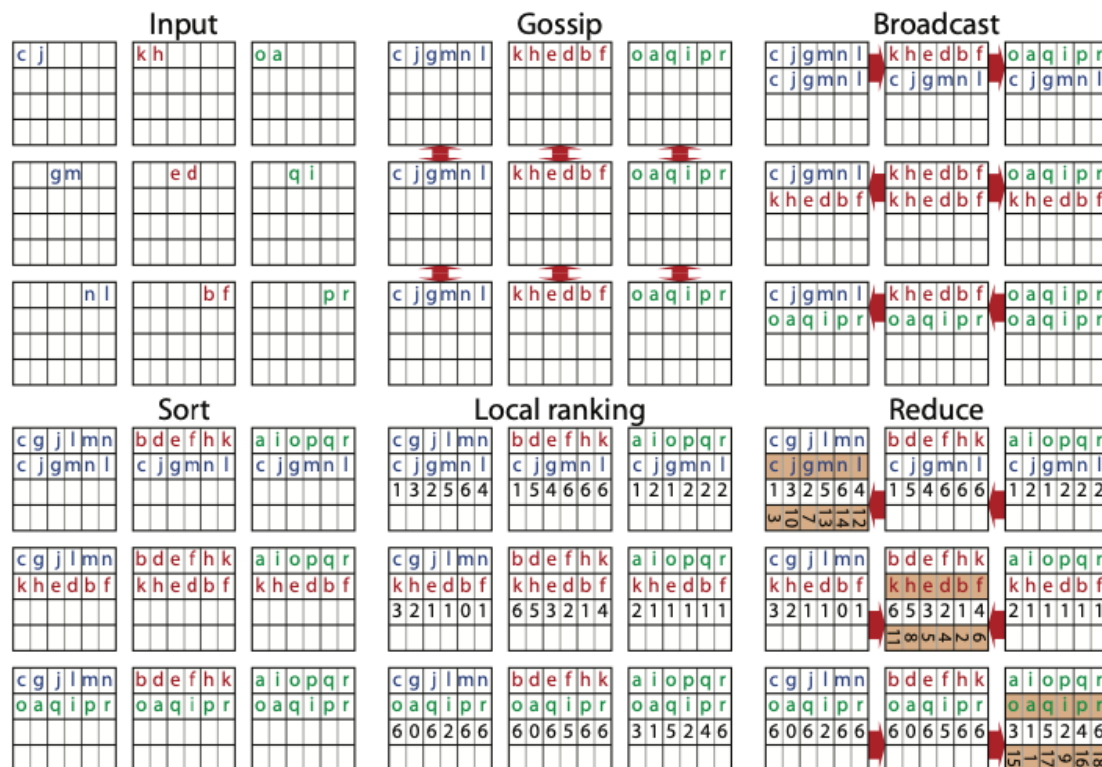
- Elabore un PRAM del algoritmo y determine la complejidad teórica del mismo
- Ejecute mediciones de tiempo con el código y compárelas con la teórica, para distinto número de procesos y tamaño del problema
- Desarrolle un software de análisis de performance que permita obtener métricas de performance. El código debe generar la representación gráfica de las métricas y llegar a conclusiones sobre la escalabilidad del mismo.

Bibliografía

[1] heat

5 Ordenamiento eficiente por ranking

Se pide desarrollar un código en paralelo para el algoritmo de ordenamiento por ranking visto en clase



El proyecto debe contener lo siguiente:

- Elabore un PRAM del algoritmo y determine la complejidad teórica del mismo
- Ejecute mediciones de tiempo con el código y compárelas con la teórica, para distinto número de procesos y tamaño del problema
- Desarrolle un software de análisis de performance que permita obtener métricas de performance. El código debe generar la representación gráfica de las métricas y llegar a conclusiones sobre la escalabilidad del mismo.

Bibliografía:

- [1] RankingSort code

6 Evolución galáctica

Simulaciones de evolución galáctica permiten predecir teóricamente el comportamiento dinámico de estrellas en galaxias y cúmulos globulares. El modelo requiere conjuntos de objetos (N



estrellas) de una cantidad muy grande, billones de estrellas en la realidad, que exigen una simulación lo más realística posible, pero cuyos resultados sean accesibles en un tiempo medido.

Esta propuesta se basa en el trabajo hecho por el grupo Astrofísica de la Universidad de Heidelberg, Alemania. Para ello, se cuenta con el código híbrido de NCuerpos PhiGPU [1].

El proyecto debe contener lo siguiente:

- a) Elaborar un PRAM del algoritmo basandose en el algoritmo de NCuerpos discutido en clase y determinar la complejidad teórica del mismo
- b) Ejecutar mediciones de tiempo con el código y compararlas con la teórica, para distinto número de procesos y tamaño del problema. El código cuenta con un Makefile que se ejecuta con: `make cpu-4th` Para modificar el tamaño del problema (cantidad de elementos) utilice en código `gen-plum.c` en la misma carpeta, modificando la variable N
- c) Desarrolle un software de análisis de performance que permita obtener métricas de performance. El código debe generar la representación gráfica de las métricas y llegar a conclusiones sobre la escalabilidad del mismo.

Bibliografía

- [1] phiGPU code
- [2] Paper2011

7 Indicaciones importantes para la entrega del proyecto

7.1 Informe parcial de proyecto

Se pide realizar un informe del avance del proyecto, que contenga la siguiente información:

- **Nombre del proyecto**, y de los **integrantes**
- **Porcentaje de participación** de cada integrante en el proyecto (de 0 a 100 %, 100 es que ha contribuido en forma equitativa al resto)
- **Introducción:** Describa el proyecto elegido y la solución planteada
- **Método (punto 1 de la rúbrica):** Describa el algoritmo usado (PRAM). Describa las métricas solicitadas (Tiempo de ejecución, Speedup, Eficiencia)
- **Bibliografía (punto 3 de la rúbrica):** Describa las fuentes utilizadas y referenciadas en el documento (e.g. IA). Describa su impacto en el desarrollo del proyecto.

7.2 Rúbrica (entrega parcial)

Se utilizará la siguiente rúbrica para la calificación del avance de proyecto

Criterio	Logrado	Parcialmente Logrado	No Logrado
1. Desarrollo de PRAM y métricas	El diseño del algoritmo es adecuado. Las métricas corresponden al PRAM (6 pts)	Existen algunos errores menores en el diseño del algoritmo y determinación de las métricas (3 pts)	Existen muchos errores en el diseño del algoritmo y métricas (1pt).
2. Uso correcto de fuentes externas	Justifica el uso de fuentes externas (publicaciones, charlas, IA) (4 pts)	Justifica solo parcialmente el uso de fuentes externas (2 pts).	No justifica el uso de fuentes externas (1pt).
3. Presentación escrita (informe) y trabajo en grupo	Describe los puntos del informe en forma ordenada y satisfactoria (4 pts)	No describe en forma adecuada todos los puntos del informe (2 pts).	No describe en forma adecuada ningún punto del informe (1pt).
4. Presentación oral	Muestra dominio del contenido del proyecto durante presentación (6 pts)	No se demostró un dominio total del contenido del proyecto durante la presentación (3 pts).	No se presentó conocimiento del proyecto durante la presentación (1pt).

7.3 Informe final de proyecto

Se pide realizar un informe de los resultados del proyecto, que contenga la siguiente información:

- **Nombre del proyecto**, y de los **integrantes**
- **Porcentaje de participación** de cada integrante en el proyecto (de 0 a 100 %, 100 es que ha contribuido en forma equitativa al resto)
- **Introducción:** Describa el proyecto elegido y la solución planteada
- **Método (punto 1 de la rúbrica):** Describa el algoritmo usado (PRAM) y/o el código.
- **Resultados y análisis (punto 2 de la rúbrica):** Describa y analice los resultados obtenidos y plantee mejoras a la solución.
- **Bibliografía (punto 3 de la rúbrica):** Describa las fuentes utilizadas y referenciadas en el documento (e.g. IA). Describa su impacto en el desarrollo del proyecto.

El proyecto se subirá a Canvas (Proyecto HPC)

Presente el documento de proyecto en formato **.pdf**. Puede convertir un documento a pdf o usar una plantilla de Plantillas Latex . No olvide adjuntar el **código de programación** o **link a Github**.

7.4 Rúbrica (entrega final)

Se utilizará la siguiente rúbrica para la calificación de la entrega final del proyecto

Criterio	Logrado	Parcialmente Logrado	No Logrado
1. Desarrollo de código/PRAM	El diseño del algoritmo y/o código es adecuado (2 pts)	Existen algunos errores menores en el diseño del algoritmo o funcionamiento del código (1 pt)	Existen muchos errores en el diseño del algoritmo y/o funcionamiento del código (0.5pts).
2. Resultados y Análisis	Desarrolla correctamente las preguntas de análisis de resultados (4 pts)	Desarrolla parcialmente las preguntas de análisis de resultados (2 pts).	Desarrolla deficientemente las preguntas de análisis de resultados (1pt).
3. Uso correcto de fuentes externas	Justifica el uso de fuentes externas (publicaciones, charlas, IA) (4 pts)	Justifica solo parcialmente el uso de fuentes externas (2 pts).	No justifica el uso de fuentes externas (1pts).
4. Presentación escrita (informe) y trabajo en grupo	Describe los puntos del informe en forma ordenada y satisfactoria (4 pts)	No describe en forma adecuada todos los puntos del informe (2 pts).	No describe en forma adecuada ningún punto del informe (1pt).
5. Presentación oral	Muestra dominio del contenido del proyecto durante presentación (6 pts)	No se demostró un dominio total del contenido del proyecto durante la presentación (2 pts).	No se presentó conocimiento del proyecto durante la presentación (1pt).